

ComicCraft: AI Comic Story Creator using Gemini Models

Project Description:

ComicCraft harnesses Generative AI to turn a short story idea into a finished, illustrated comic. The platform includes an Outline module, which asks Gemini Flash for a five-panel plan in structured JSON, a Story module, which asks a second Gemini model for a caption and narration for every panel, and an Image module, which draws each panel locally with Stable Diffusion v1.5. A layout builder merges text and images into a preview page and a one-page-per-panel PDF.

Built with FastAPI and Jinja2, the platform runs on a single laptop (Apple Silicon MPS, CUDA or CPU), keeps API keys in a git-ignored .env file and needs only a free Gemini API key. ComicCraft empowers students, teachers and creators to share visual stories without any drawing skills.

Scenarios

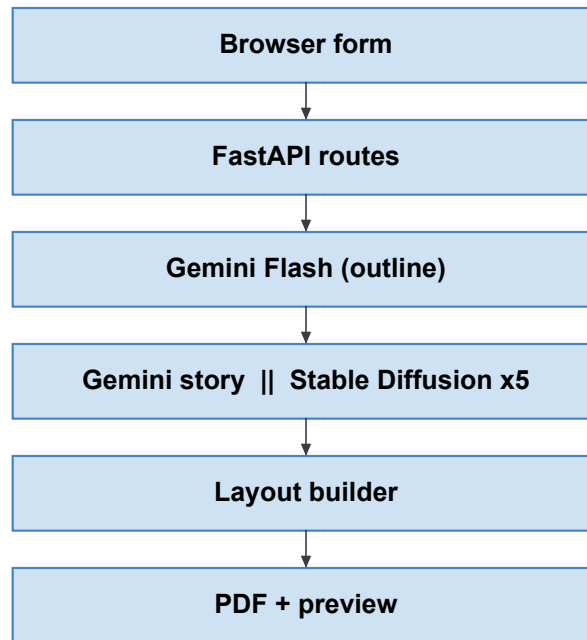
Scenario 1: A student types "A brave fox explores an enchanted forest", names the character Rio and chooses Forest, Light-hearted and Anime. ComicCraft returns five panels with captions, narration and a PDF in under two minutes.

Scenario 2: A teacher prepares a science lesson, chooses Space, Dramatic and Comic book. The generated story and images become a handout without any commissioned artwork.

Scenario 3: A content creator wants a quick social post, picks City, Funny and Pixel art and downloads the PDF to share, regenerating with a different style until satisfied.

Technical Architecture:

Description: ComicCraft uses a modular layered architecture. The browser shows Jinja2 pages; the FastAPI backend validates the form, builds prompts and orchestrates the pipeline. Gemini Flash returns the outline. In parallel, a second Gemini model writes the story and Stable Diffusion v1.5 draws five images. The layout builder merges the results and fpdf2 writes the PDF. Generated files are stored on disk; no database is used.



(No database is used in this project, so no ER diagram is required.)

Pre-requisites:

- **Python programming:** Python documentation (python.org)
- **FastAPI and Jinja2:** FastAPI documentation (fastapi.tiangolo.com)
- **Gemini API familiarity:** Google AI Studio and Gemini API documentation
- **Stable Diffusion and diffusers:** Hugging Face diffusers documentation
- **HTML and CSS:** MDN Web Docs
- **Version control with Git:** Git documentation
- **Development environment:** Python 3.14 virtual environment, requirements.txt

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Create a Gemini API key and store it securely in .env.
- Activity 1.2: Select the models: Gemini Flash for the outline, another Gemini model for the story, Stable Diffusion v1.5 for images.
- Activity 1.3: Define the architecture and the data passed between modules.
- Activity 1.4: Set up the virtual environment and install dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop generate_outline, generate_story and generate_image.
- Activity 2.2: Develop the layout builder and the PDF exporter.

Milestone 3: App.py Development

- Activity 3.1: Write main.py and routes.py with the form, JSON and test routes and shared create_comic().

Milestone 4: Frontend Development

- Activity 4.1: Design the glass-style home page with pill options and loading state.
- Activity 4.2: Create the preview and export templates with Jinja2.

Milestone 5: Testing and Deployment

- Activity 5.1: Write pytest tests with mocked Gemini and image calls.
- Activity 5.2: Run locally with Uvicorn and verify the full pipeline.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone we choose the generative models and set up the project. Text generation uses the Gemini API; image generation runs locally so that no per-image cost or quota applies.

Activity 1.1: Create a Gemini API Key

- **Step 1:** Open Google AI Studio and sign in with a Google account.
- **Step 2:** Create an API key.
- **Step 3:** Create a .env file in the project root and add the key:

```
GEMINI_API_KEY=your_gemini_api_key_here  
# Optional overrides  
# GEMINI_FLASH_MODEL=... GEMINI_PRO_MODEL=...
```

- **Step 4:** The app loads the key at start-up with python-dotenv (load_dotenv() in app/main.py).
- **Important:** never commit the key. .env is listed in .gitignore; only .env.example is committed.

Activity 1.2: Select the Models

- **Gemini Flash** for the outline: fast and supports JSON output (response_mime_type).
- **A second Gemini model** for the story, because the free tier allows only 20 requests per day per model; splitting the two calls doubles the number of comics per day.
- **Stable Diffusion v1.5** for panel images: runs on 16 GB hardware in fp16 and is free to run locally.

Activity 1.3: Define the Architecture

Each module has one job: routes.py orchestrates; gemini_flash.py and gemini_pro.py talk to Gemini; image_generator.py owns the cached Stable Diffusion pipeline; layout_builder.py merges data; exporters.py writes the PDF.

Activity 1.4: Set Up the Environment

```
python -m venv env  
source env/bin/activate  
pip install -r requirements.txt
```

Milestone 2: Core Functionalities Development

In this milestone we build the AI and data functions that make up the comic pipeline.

Activity 2.1: AI Modules

- **generate_outline** builds a prompt from the idea, character, setting, tone and style, requests JSON and normalises it to at most five panels with panel, title, scene and image_prompt.
- **generate_story** sends the outline and asks for Panel / Caption / Narration blocks.
- **generate_image** runs Stable Diffusion for 25 steps and saves a PNG with a sanitised file name, returning its URL.
- The pipeline is created once (_get_pipe) and warmed up in a background thread at start-up.

Activity 2.2: Layout and PDF

- **build_comic_layout** parses the story with regular expressions and merges it with images and outline; it tolerates missing captions.
- **save_pdf** writes one page per panel using DejaVu fonts for Unicode text.

Milestone 3: App.py Development

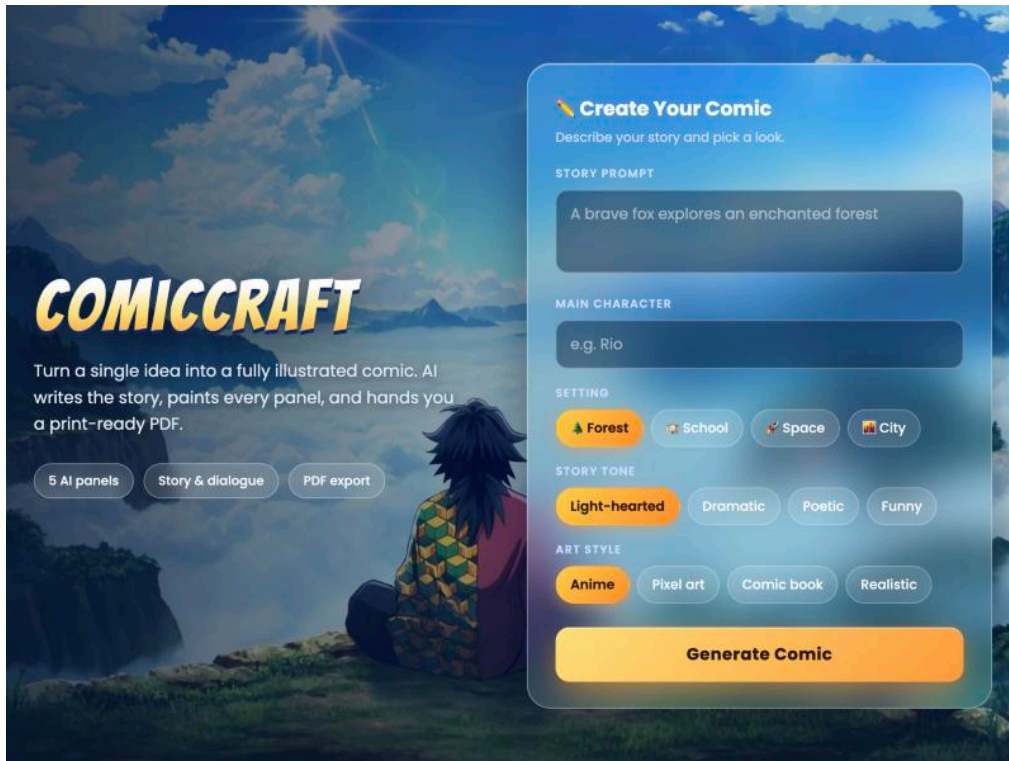
main.py creates the FastAPI app, mounts /static, includes the router and starts the warm-up thread. routes.py defines:

Route	Method	Purpose
/	GET	Home form
/generate	POST	Run pipeline and show the preview
/generate-comic/json	POST	Same pipeline, JSON result
/export-success	GET	Download confirmation
/test-image	GET	Single test image

create_comic() generates the outline, then runs the story and the five images in a ThreadPoolExecutor, builds the layout and saves the PDF. Any failure is returned as a readable HTTP 500.

Milestone 4: Frontend Development

The UI uses Jinja2 templates with a background image, glass-style card, Bangers and Poppins fonts and pill-style selectors for setting, tone and art style. The Generate button shows a loading label while the request runs. The preview page shows each panel with its title, image, caption and narration.



The screenshot displays the 'COMICCRAFT' home page. The background features a sunset scene with a person sitting on a cliff. The 'COMICCRAFT' logo is prominently displayed in a stylized, glowing font. Below the logo, a brief description states: 'Turn a single idea into a fully illustrated comic. AI writes the story, paints every panel, and hands you a print-ready PDF.' Three buttons are visible: '5 AI panels', 'Story & dialogue', and 'PDF export'. On the right, a 'Create Your Comic' form is shown with the following sections:

- Create Your Comic**
Describe your story and pick a look.
- STORY PROMPT**
A text input field containing 'A brave fox explores an enchanted forest'.
- MAIN CHARACTER**
A text input field containing 'e.g. Rio'.
- SETTING**
Four pill-style buttons: Forest (selected), School, Space, and City.
- STORY TONE**
Four pill-style buttons: Light-hearted (selected), Dramatic, Poetic, and Funny.
- ART STYLE**
Four pill-style buttons: Anime (selected), Pixel art, Comic book, and Realistic.
- Generate Comic**
A large orange button at the bottom of the form.

Home page: Create Your Comic form

Your Comic

Panel 1: The Threshold of Adventure



Preview page with generated panels

Milestone 5: Testing and Deployment

Activity 5.1: Automated tests. The suite has 24 pytest tests (FastAPI TestClient). Gemini and Stable Diffusion are replaced by fakes, so tests run offline in seconds. They cover outline normalisation and limits, invalid JSON, prompt contents, story parsing, layout, image file naming, PDF writing, all routes and the quota error path.

```
pip install -r requirements-dev.txt
PYTHONPATH=. python -m pytest -q tests
# 24 passed
```

Activity 5.2: Run locally. Start the server and open the home page:

```
uvicorn app.main:app --port 8000
# open http://127.0.0.1:8000
```

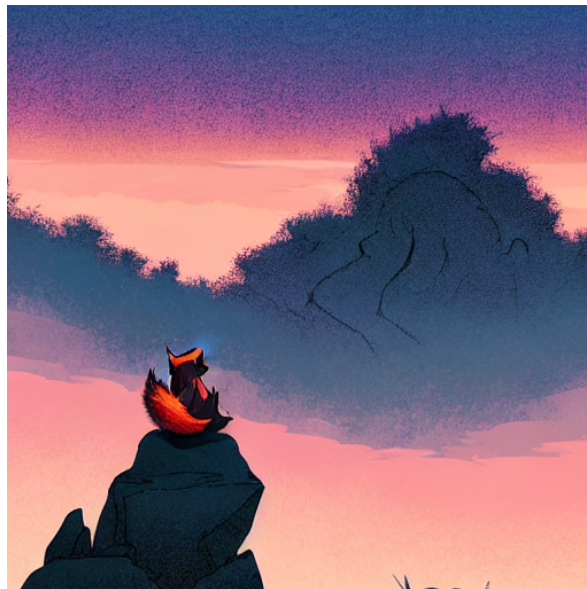
Measured on Apple M5 (16 GB): about 12.4 s per panel image after warm-up and 104 s from form submit to preview (5 panels and PDF).

Exploring the Website's Web Pages

Home / Generator page. The page opens with the title and a glass-style form.

Input section. The user enters the story idea and main character, then selects setting (Forest, School, Space, City), tone (Light-hearted, Dramatic, Poetic, Funny) and art style (Anime, Pixel art, Comic book, Realistic).

Results section. After generation the preview shows five panels. The export page confirms the PDF download.



A generated panel (Stable Diffusion v1.5)

Panel 1: The Threshold of Adventure



Rio, a small orange fox wearing an oversized green scarf and steampunk-style goggles, stands confidently at the entrance of the Whispering Woods. The trees are tall with shimmering silver leaves, and a path of soft moss leads into the depths of the vibrant greenery.

Caption: The threshold of the Whispering Woods awaits.

Rio adjusted his goggles and wrapped his oversized green scarf tighter against the morning chill. "Today is the day," he chirped to the silver trees, his tail twitching with anticipation. He took a bold step forward, ready to uncover the secrets hidden within the glowing canopy.

A page of the exported PDF

Milestone 6: Conclusion

ComicCraft shows that a complete creative workflow (idea, outline, story, art, layout and PDF) can run from one form using free-tier Gemini and a local image model. Limits remain: the free Gemini quota, no character consistency between panels and about 100 s generation time. Planned next steps are speech bubbles, character consistency, a progress indicator, CI for the test suite and Docker deployment.

Team

Member	Role
Fathima Fahmiya S	Backend Developer
Nivriti Muthu Vairavan	Team Leader, AI & Image Engineer
Mounika K M	Frontend & UI/UX Developer
Tharun P	Testing & QA Engineer
Yuvaraj B	Documentation & Demo Coordinator